

Code Was Never the Problem

- How AI Sorts Your Team's Practices

Peter Gfader - Silo Thinking Breaker

Writing code is now cheap, which changes everything except the code.

This interactive session takes one uncomfortable claim seriously: **good engineering practices don't die under AI, they get sorted.**

Whatever was merely a proxy for "more code, faster" loses its justification.

Whatever builds and carries an understanding of a system grows more valuable — because of AI, not despite it.

We'll trace this through three things every team wants right now: **correct working code**, **costs under control**, and **enough experienced developers**, with practical theories like Jevons' Paradox and Peter Naur's "Programming as Theory Building".

With interactive elements and concrete things you'll do differently on Monday.



1.Trainer

a.Tools, Methoden

2.Erfahrungen-Teiler

a.Methodisch

b.Engineering

3.Scrum Master

a.Schwierigste = Welches Problem ist es wert zu lösen

4.Systemisch-Denker

a.Find the True Reason

b.Lass uns den Sinn im Unsinn finden



Software Crafters Zürich

★★★★★ 4.6 • 694 ratings

📍 Zürich, Switzerland

👤 2,907 members



Technical Samman Coach

Connect With Us



<https://twitter.com/peitor>



<https://www.linkedin.com/in/petergfader/>



peter@beyond-agility.com



Google Group



<https://beyond-agility.com>



Scan me



AI is the end of programming.

vs software
engineering

AI is the end of the complicated parts of programming

*lets start the beginning of an discussion
about **what in programming**
was never actually complicated,
but always complex.*

Big challenge
for humans



<https://www.youtube.com/watch?v=TfSD35kNuJ4>

What did you hear?



Green Needle vs Brainstorm

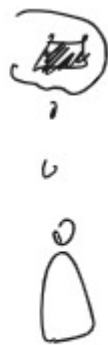
<https://www.youtube.com/watch?v=qXxV2C1ri2k>

Semantic Diffusion

What about Domain Concepts

Quote / Offer

Account Number / PartnerId



we are aligned

Change Curve

OPINATED



With previous technological shifts, the rule was:

"Learn the new stuff and put your skills to use."

With AI, that doesn't work. It's not a new platform, not a new language, not a new paradigm.

AI raises the question of what it even means to be a good developer.

- Written code manually yourself
- Refactored code manually
- Instruct 1 agent
- TDD with an agent
- More than 2 agents in parallel
- Reviewing some of the generated code
 - 0 code reviews
- Non-Dev vibing software



Reading? → put in context memory
output. and

Babysitting? ☺

How long do you go away?

↳ semantic anchor
skill?
maintenance audit

As of early 2026,

Tolan.co.uk

- 85% of professional developers regularly use AI Coding Agents,
- 51% use them daily, and an
- estimated 41% of all new code is AI-generated

<https://www.kaggle.com/whitepaper-the-new-SDLC-with-vibe-coding>

[scrum.org/](https://scrum.org/aps-sd)
[aps-sd](https://scrum.org/aps-sd)

You are not
allowed to
write any code
yourself

What are good engineering practices?

In
pairs

Discuss

Top 3

TDD

MODULARIZATION

ANALYSIS → WHY

CONSIDER OUR LIBS

GO40 CASE

UNDERSTAND VSOL

PROBLEM

What are good engineering practices?

D.D.D. 

SUBDOMAINS / BOUNDED
CONTEXTS

BEHAVIOUR TESTS ON BOUNDARY
EVALS

ADD PER BOUNDARY

↳ INSIDE THE RIGHT
APP ARCHITECTURE

"FITNESS FCOS"

WHAT
HOW

CLEAN / HEALTHY CODE

CODE SCENING

TOKENS

BUGS

YOUR CODE IS
YOUR PROMPT

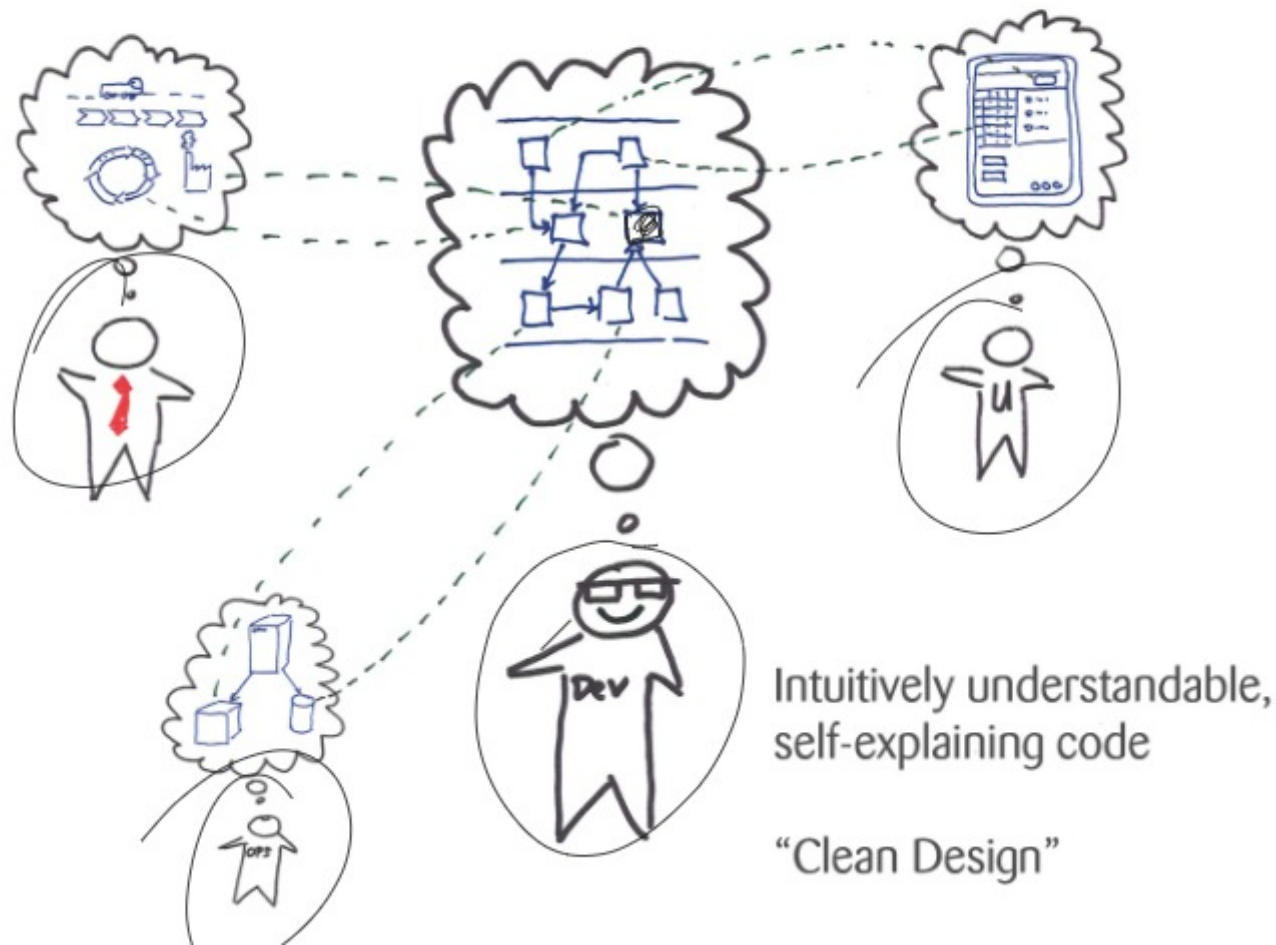
Which one deals
with Semantic
Diffusion?

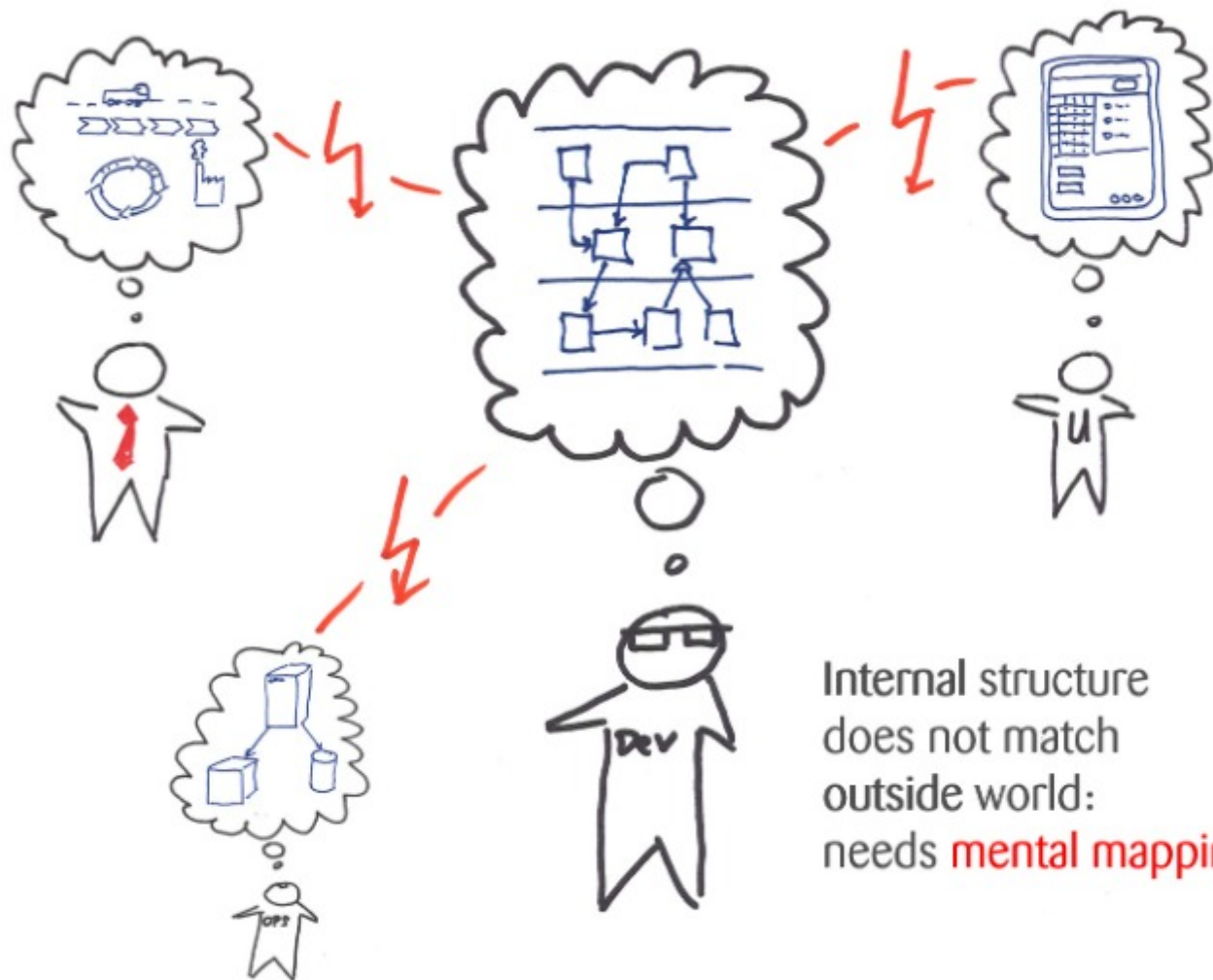
Programming as Theory Building

Peter Naur

1985

Peter Naur's classic 1985 essay "Programming as Theory Building" argues that a program is not its source code. A program is a shared mental construct (he uses the word theory) that lives in the minds of the people who work on it. If you lose the people, you lose the program. The code is merely a written representation of the program, and it's lossy, so you can't reconstruct a program from its code.



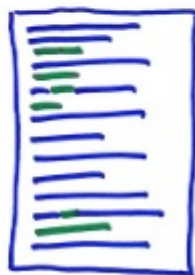
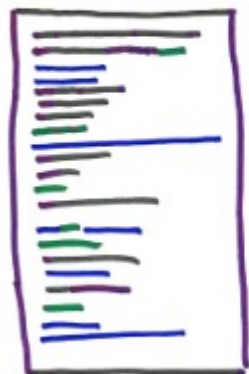
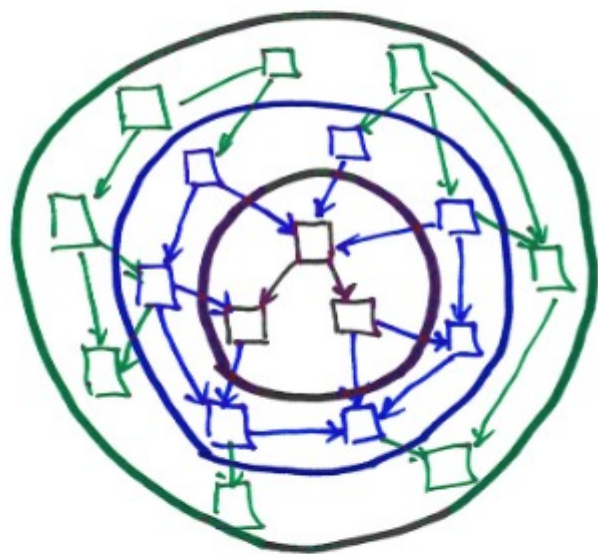


Internal structure
does not match
outside world:
needs **mental mapping**

Fear to change leads to annual rings in code

Before LLMs 20%

Now 2%
COPY PASTE

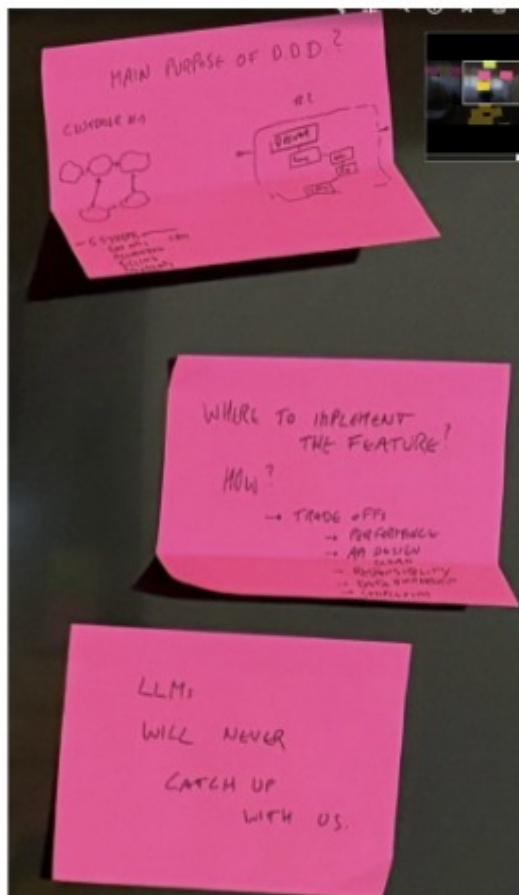


— Initiale Funktionalität
— Erste Erweiterungswelle
— Zweite Erweiterungswelle

DDD -> Systems Get Bigger

Main Purpose of DDD?

Main Purpose Of Good Architecture?



CD V Isolation
DDD

Kinds of Coupling

Type	Effect
Operational	Consumer cannot run without the provider
Development	Changes in producer and consumer must be coordinated
Semantic	Change together because of shared concepts
Functional	Change together because of shared responsibility
Incidental	Change together for no good reason. (E.g., breaking API changes.)

(+) Added Peter

„Temporal Coupling“ is also the reason why I need access to the git history ->

Indicator for: „Code that changes together“

(Temporal Coupling is very painful and a special case of Functional and Semantic Coupling)

Michael Nygard has a good starting point to “Kinds of Coupling”

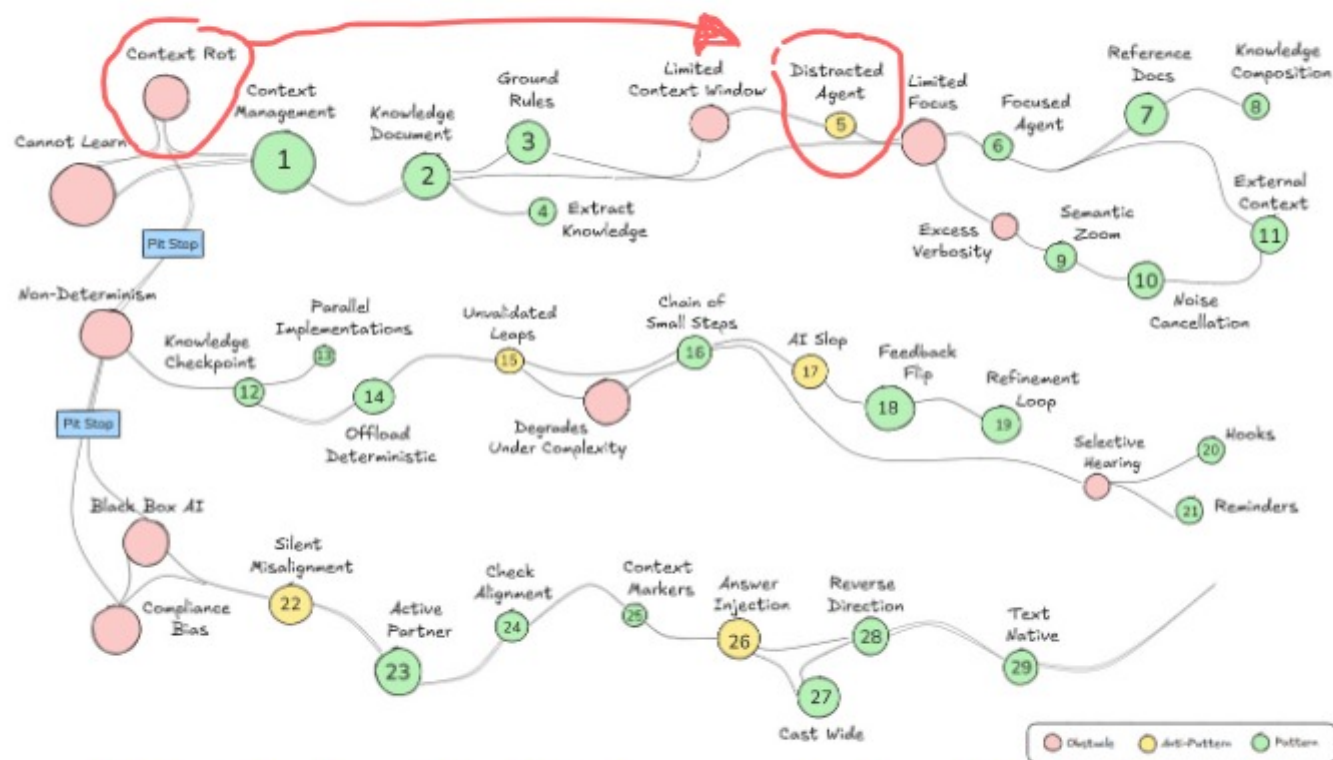
<https://www.youtube.com/watch?v=esm-1QXtA2Q>

Types of coupling and my #pain assessment:

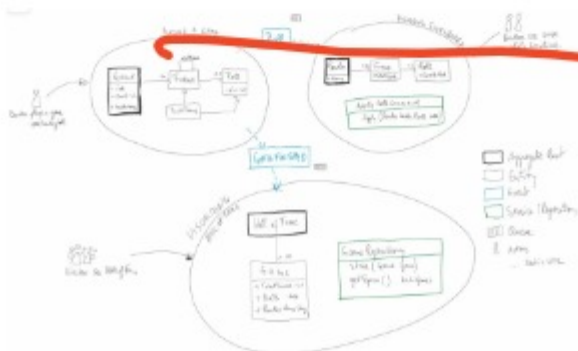
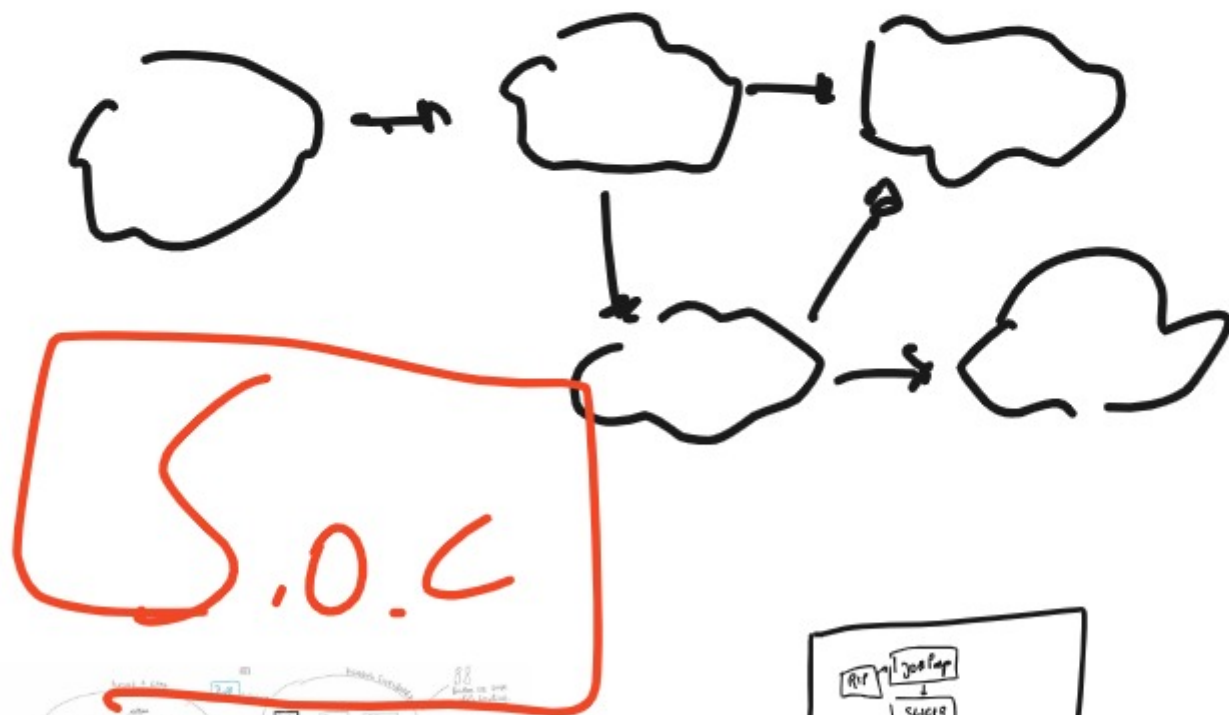
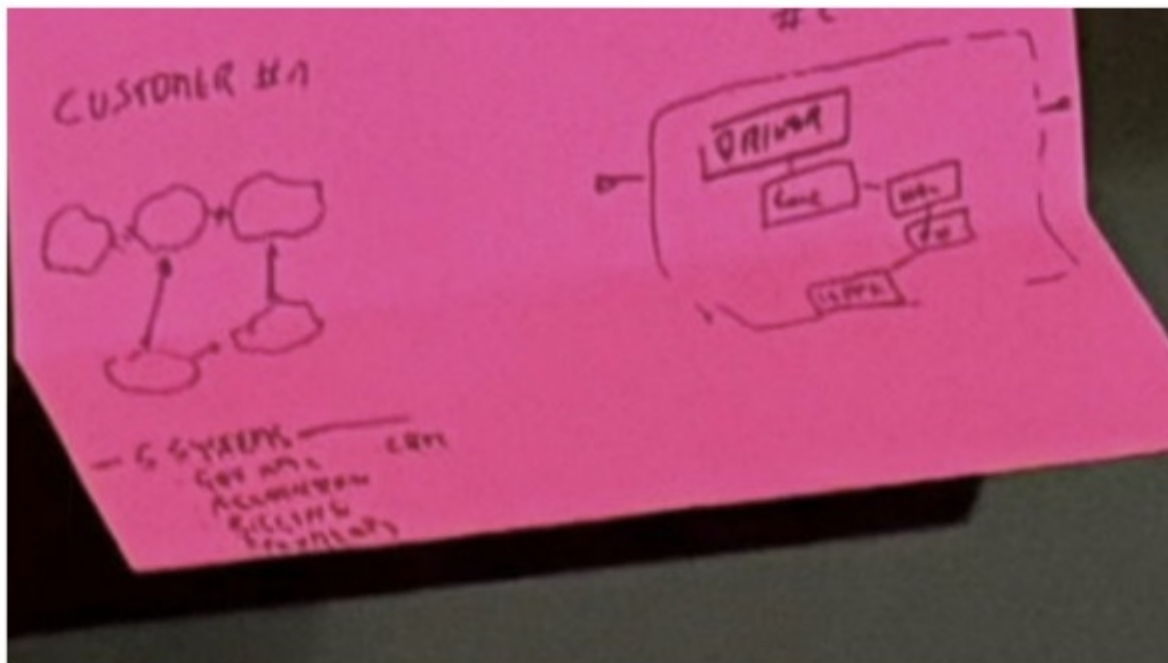
- Operational
 - Runtime + health monitoring help here
- Development
 - Coordinated changes are needed here
 - That's what our baseline is good for, but **when do we realize that?**
- Semantic
 - Dangerous, very dangerous. You have to keep this under control
 - OK in a bounded context
- Functional
 - If within a team = good and easy to manage.
 - Can be cumbersome across teams.
- ~~Incidental (not really a separate type)~~
- (+) Added Peter: „Temporal Coupling“ is also the reason why I need access to the git history -> Indicator for: „Code that changes together“
 - *(Temporal Coupling is very painful and a special case of Functional and Semantic Coupling)*

Context window is limited

- **fixed size**
- first in, first out: oldest content gets **erased**
- **everything** competes for limited space



<https://lexler.github.io/augmented-coding-patterns/>



How?

EventStorming

DDD

Subdomains



Bounded
Contexts

Per Bounded Context

Code coverage trends.

Linters in pipeline

Mutation testing inside hexagon

ATDD



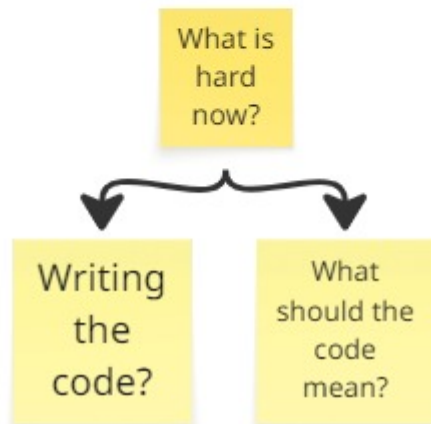
Design the Interface

Generate Implementation

Integration Events

Across Teams

Leaks into your subdomains / bounded context



Event language

- CustomerUpdated?
- CustomerEmailAddressChanged?
- CustomerProfileUpdated?

Design questions

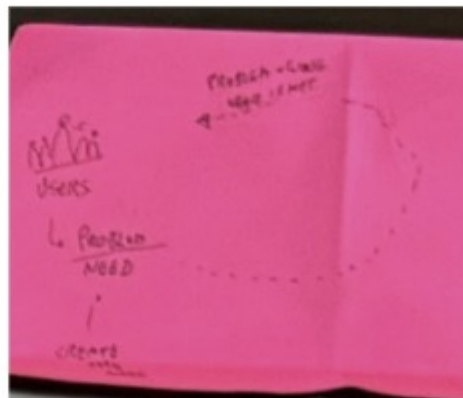
- Who owns the data?
- Which boundary is crossed?
- What should the agent copy?

CRUD
leaks in



Business
People talk
in CRUD?

<https://www.youtube.com/watch?v=rAWvjy0MuvM>



Build The Right Thing

<https://www.youtube.com/watch?v=3ImClkPKQ0k&t=580s>

Complicated VS Complex

- Causality
- Rules
- Machines

- Surprises
- Principles

Means

Die
Kompliziertheits-
Anteile werden
automatisierbar



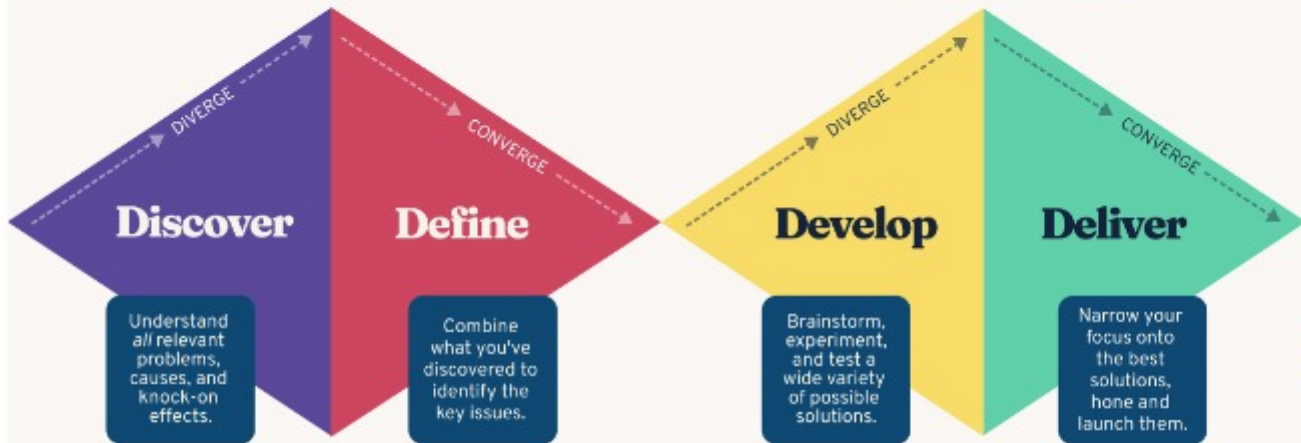
die Komplexitäts-
Anteile bleiben – und
damit verschiebt sich
der eigentliche Beitrag
des Engineers in den
Komplex Bereich.

The double diamond design process

A four-step process for designing solutions to complex problems.

What's the problem?

What's the solution?



CC BY 4.0: The Design Council

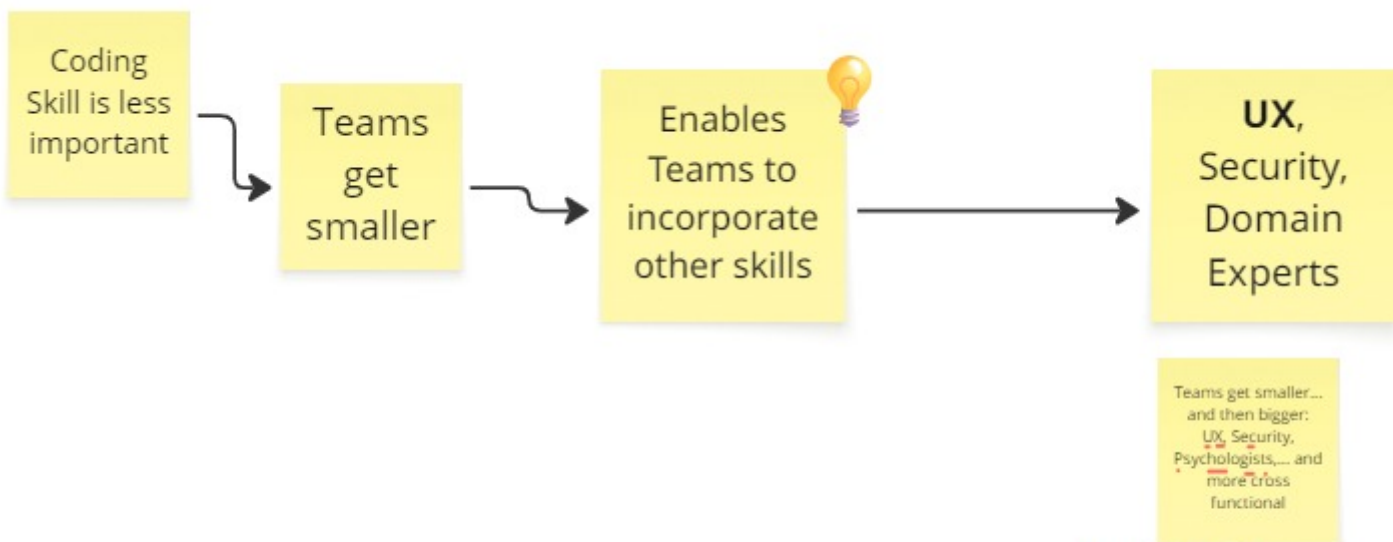
BiteSize Learning

CODE WAS NEVER THE PROBLEM
WHY NOW?

Percentage of Complex Problems?
Complicated Problems?

How to deal with those?

How does Team+Characteristics change with AI 🤖



INFRASTR.
10- & SUIBLO

#1 A skill? Practice?

QA Team, Analysis Team, ...

#2 An island? a subcomponent?

OCR Invoice Scanner

#3 Integrate with other to deliver the business capability.

TIPS

Keep studying **system** design.

Architecture becomes even more important when code becomes cheap.

Learn **databases** properly. Indexes, transactions, isolation, replication and schema design still decide whether systems survive scale.

Understand **networking**. TCP, HTTP, DNS, retries and timeouts explain a surprising number of production issues.

Learn **concurrency**. Race conditions, deadlocks and ordering bugs become expensive very quickly.

... SECURITY ... + EVOLUTION

TIPS FOR
18+ OLD
SOFTWARE ENGINEERS

Grounded
Way
of Learning

step - step

ANOTHER
GOOD
STEP

Roadmaps
roadmap.sh

Internationally
Recognized
Certification
Course

Tech skills
And
Social skills

Learn about
Communication

Learning to
Learn

Learn to
Learn

Where am I and how
to get to where I want to go
from here?

T

→ SHAPED

COMB

ANALYSIS

Transfer

Competitive
programming
platforms
(LeetCode)

- Understand Motivation
is the most important!
- Courses are not enough
to motivate yourself
- Focus on engineering fundamentals
- Get started, now and
don't let it go for you

Find something
that you like

Follow your
dreams / interests
(Just do it!)

DO SIDE PROJECTS
AND LEARN by
doing
open source, community
helpful, have fun

Surround yourself
with ppl that
are aligned

Critical
Thinking

Growth on
your own time
org vs. you!

Don't fall in the
Dunning Kruger curve

LEARN
BASICS*
PRINCIPLES

OUT
OF
THE
BOX

+ maybe a intermediate

FUNDAMENTALS

Transfer
of Knowledge
How to get

Continuous Delivery

TIP

Ship products all the way.

Deployment, monitoring, incidents and maintenance teach lessons demos never will.

Monitoring, alerting, observability, feature flags, dark launching, smoke testing, acquiring prod-like test data

UX

Architecture



Data Science

Security

Engineers no longer create artifacts themselves;
instead, they orchestrate and validate their generation.

3 new skill groups:

1. Orchestration (agents/context architecture),
2. Validation (checking intent fidelity),
3. Evaluation (continuous improvement of the agent architecture).

Core engineering skills remain necessary but are being applied in new ways.

Which craftsmanship skills are still relevant in the age of AI?
Do you have any tips for practicing those skills?

A: ❤️ for users

- DRY
- refining requests
 - break down larger things
 - slicing

Become a Domain Expert

The language

The quirks

The people

The Users

The Market

THE REAL WORLD

- Event Storming bringt dich in einen Raum mit echten Menschen
- Tests bringen dich in Kontakt mit tatsächlichem Verhalten statt vermutetem
- Continuous Delivery bringt dich in Kontakt mit echten Nutzern
- Domain-Expertise aufbauen heisst, dich der Wirklichkeit der Fachdomäne auszusetzen

"GOOD PRACTICES
CONNECT YOU TO REAL WORLD"



WHAT IS
YOUR TAKE AWAY?

How Do
You Learn
Faster?

Peter Gfader - Silo Thinking Breaker



Programming as Theory Building

<https://pablo.rauzy.name/dev/naur1985programming.pdf>

Michael Nygard has a good starting point to "Kinds of Coupling"

<https://www.youtube.com/watch?v=esm-1QXtA2Q>

The key concept in computer science and how it relates to ATDD (Peter Gfader)

<https://www.youtube.com/watch?v=3lmClkPKQ0k&t=580s>

Training: scrum.org/aps-sd

"You are not allowed to WRITE code"

Augmented Coding Patterns

A collection of emerging patterns, anti-patterns and obstacles for effective AI-augmented software development

<https://lexler.github.io/augmented-coding-patterns/>

Boundary Events -> AI Can Write Code. How Do We Keep It Clean?

<https://www.youtube.com/watch?v=rAWvjy0MuvM>

Code Reviews is Not a New Problem

https://www.youtube.com/watch?v=fHWFF_pnqDk

 Rate your Tests with the Claude Code Farley Plugin

https://github.com/mse-online/farley_score_plugin

Connect With Us



<https://twitter.com/peitor>



<https://www.linkedin.com/in/petergfader/>



peter@beyond-agility.com



Google Group



<https://beyond-agility.com>



Scan me